

Frontend Integration Guide - Marketing Hub API

Complete API documentation for frontend engineers

Last Updated: December 26, 2025

API Version: 1.0.0

Table of Contents

1. [Quick Start](#)
2. [API Architecture](#)
3. [Authentication Setup](#)
4. [Core Workflows](#)
5. [API Reference](#)
6. [Common Integration Patterns](#)
7. [Error Handling](#)
8. [TypeScript Types](#)
9. [Code Examples](#)

Quick Start

Base URL

```
Development: http://localhost:8000
Production: https://your-api-domain.com
```

Interactive API Docs

```
Swagger UI: http://localhost:8000/docs
ReDoc: http://localhost:8000/redoc
```

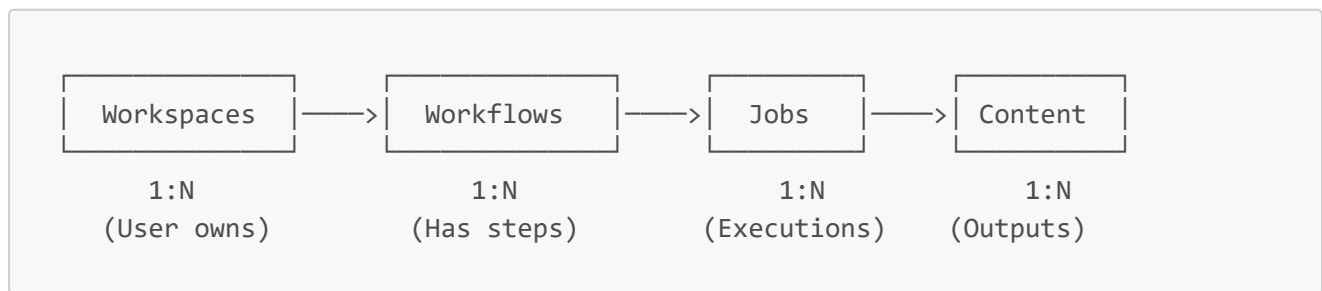
Health Check

```
GET /health
# Response: {"status": "healthy"}
```

API Architecture

System Overview

The Marketing Hub API follows a workflow orchestration pattern (MCP - Master Control Program):



Key Concepts

1. **Workspaces**: Container for user's marketing operations
2. **Workflows**: Reusable automation definitions with sequential steps
3. **Jobs**: Individual executions of workflows (tracks progress)
4. **Content**: Tool outputs stored for approval/reuse

Authentication Setup

Current Implementation

The API currently uses **Supabase authentication**. Include the Supabase token in requests:

```
const headers = {
  'Content-Type': 'application/json',
  'Authorization': `Bearer ${supabaseToken}`,
  'apikey': process.env.SUPABASE_ANON_KEY
};
```

CORS Configuration

CORS is enabled for all origins. In production, this should be restricted:

- Allowed Methods: GET, POST, PUT, PATCH, DELETE
- Allowed Headers: All
- Credentials: Enabled

Core Workflows

Workflow 1: First-Time User Setup

```
// 1. User signs up via Supabase Auth
const { user } = await supabase.auth.signUp({ email, password });

// 2. Create workspace
const workspace = await fetch('/api/workspaces/', {
```

```

method: 'POST',
headers,
body: JSON.stringify({
  owner_user_id: user.id,
  name: "My Marketing Workspace"
}))
});

```

Workflow 2: Create and Run Automation

```

// 1. Create workflow
const workflow = await createWorkflow({
  workspace_id: workspaceId,
  name: "Daily LinkedIn Post",
  target_platform: "linkedin",
  steps: [
    {
      order: 1,
      name: "Generate Caption",
      tool_name: "caption_generator",
      config: { topic: "AI marketing", tone: "professional" }
    }
  ]
});

// 2. Run workflow
const job = await runWorkflow(workflow.id);

// 3. Poll job status
const checkStatus = setInterval(async () => {
  const status = await getJobStatus(job.job_id);
  if (status.status === 'completed') {
    clearInterval(checkStatus);
    // 4. Get generated content
    const content = await getContent({ job_id: job.job_id });
  }
}, 2000);

```

Workflow 3: Content Approval Flow

```

// 1. Get pending content for workspace
const pending = await fetch(
  `/api/content/workspace/${workspaceId}/pending`
);

// 2. Review and approve
await fetch(`/api/content/${contentId}/approve`, { method: 'POST' });

```

```
// 3. Mark as posted after publishing
await fetch(`/api/content/${contentId}`, {
  method: 'PATCH',
  body: JSON.stringify({ status: 'posted' })
});
```

API Reference

1. Workspaces API

Create Workspace

```
POST /api/workspaces/
Content-Type: application/json

{
  "owner_user_id": "uuid",
  "name": "string"
}
```

Response: 200 OK

```
{
  "id": "uuid",
  "owner_user_id": "uuid",
  "name": "My Workspace",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Get Workspace

```
GET /api/workspaces/{workspace_id}
```

Response: Same as create response

2. Workflows API

Create Workflow

```
POST /api/workflows/
Content-Type: application/json
```

```

{
  "workspace_id": "uuid",
  "name": "string",
  "description": "string (optional)",
  "target_platform": "linkedin" | null,
  "steps": [
    {
      "order": 1,
      "name": "string (optional)",
      "tool_name": "string",
      "config": {
        "key": "value"
      }
    }
  ]
}

```

Available Tools:

- **blog_generator**: Generates blog posts with cover images
- **image_generator**: Creates motivational quote images
- **caption_generator**: AI-powered social media captions
- **content_optimizer**: Improves existing content
- **hashtag_generator**: Generates relevant hashtags

Tool Configurations:

```

// Caption Generator
{
  tool_name: "caption_generator",
  config: {
    topic: "string",
    platform: "linkedin" | "twitter" | "facebook",
    tone: "professional" | "casual" | "humorous",
    include_emojis: true,
    include_hashtags: true
  }
}

// Blog Generator
{
  tool_name: "blog_generator",
  config: {
    topic: "string",
    style: "informative" | "storytelling" | "tutorial"
  }
}

// Hashtag Generator

```

```

{
  tool_name: "hashtag_generator",
  config: {
    topic: "string",
    count: 5,
    platform: "linkedin"
  }
}

// Content Optimizer
{
  tool_name: "content_optimizer",
  config: {
    content: "string",
    goal: "engagement" | "clarity" | "seo",
    audience: "professionals" | "general",
    platform: "linkedin"
  }
}

```

Response: 200 OK

```

{
  "id": "uuid",
  "workspace_id": "uuid",
  "name": "My Workflow",
  "description": "Optional description",
  "target_platform": "linkedin",
  "steps": [
    {
      "id": "step_1",
      "order": 1,
      "name": "Step Name",
      "tool_name": "caption_generator",
      "config": { /* config object */ }
    }
  ],
  "created_at": "2025-12-26T00:00:00Z",
  "updated_at": "2025-12-26T00:00:00Z"
}

```

List Workflows

```
GET /api/workflows/?workspace_id={uuid}
```

Response: 200 OK

```
[
  {
    "id": "uuid",
    "name": "Workflow Name",
    "description": "Description",
    "target_platform": "linkedin",
    "last_run_at": "2025-12-26T00:00:00Z",
    "last_run_status": "completed" | "failed" | "running" | "pending",
    "created_at": "2025-12-26T00:00:00Z"
  }
]
```

Get Workflow Details

```
GET /api/workflows/{workflow_id}
```

Response: Same as create workflow response (includes steps)

Update Workflow

```
PUT /api/workflows/{workflow_id}
Content-Type: application/json

{
  "name": "string (optional)",
  "description": "string (optional)",
  "target_platform": "linkedin" | null (optional),
  "steps": [ /* new steps array (optional) */ ]
}
```

Note: Cannot update if workflow has running jobs.

Delete Workflow

```
DELETE /api/workflows/{workflow_id}
```

Response: 200 OK

```
{
  "message": "Workflow deleted successfully"
}
```

Note: Soft delete (sets `deleted_at` timestamp)

Run Workflow ★ MOST IMPORTANT

```
POST /api/workflows/{workflow_id}/run
```

Response: 200 OK

```
{
  "job_id": "uuid",
  "workflow_id": "uuid",
  "status": "pending",
  "started_at": "2025-12-26T00:00:00Z"
}
```

Important: This endpoint returns immediately. Use the `job_id` to poll for completion.

3. Jobs API

Get Job Status

```
GET /api/jobs/{job_id}
```

Response: 200 OK

```
{
  "id": "uuid",
  "workflow_id": "uuid",
  "status": "pending" | "running" | "completed" | "failed",
  "progress": 0-100,
  "current_step": 1,
  "logs": {
    "message": "Status message",
    "step": "Current step name",
    "result": { /* tool output */ },
    "content_id": "uuid"
  },
  "started_at": "2025-12-26T00:00:00Z",
  "completed_at": "2025-12-26T00:00:00Z",
  "error": null
}
```


Status Flow:

pending → running → completed/failed

Get Jobs for Workflow

GET /api/jobs/workflow/{workflow_id}?limit=10

Response: Array of job objects (most recent first)

Polling Best Practices

```
async function waitForJobCompletion(jobId, onProgress) {
  return new Promise((resolve, reject) => {
    const interval = setInterval(async () => {
      try {
        const job = await getJobStatus(jobId);

        // Call progress callback
        if (onProgress) {
          onProgress(job.progress, job.current_step);
        }

        // Check terminal states
        if (job.status === 'completed') {
          clearInterval(interval);
          resolve(job);
        } else if (job.status === 'failed') {
          clearInterval(interval);
          reject(new Error(job.logs?.error || 'Job failed'));
        }
      } catch (error) {
        clearInterval(interval);
        reject(error);
      }
    }, 2000); // Poll every 2 seconds

    // Timeout after 5 minutes
    setTimeout(() => {
      clearInterval(interval);
      reject(new Error('Job timeout'));
    }, 300000);
  });
}
```

4. Content API

List Content

```
GET /api/content/?workspace_id={uuid}&status={status}&content_type={type}&limit=50
```

Query Parameters:

- **workspace_id**: Filter by workspace (optional)
- **job_id**: Filter by job (optional)
- **content_type**: `blog_post` | `image` | `caption` | `hashtags` | `optimized_content` (optional)
- **status**: `draft` | `approved` | `used` | `posted` (optional)
- **limit**: Max results (default 50, max 100)

Response: 200 OK

```
[
  {
    "id": "uuid",
    "workspace_id": "uuid",
    "job_id": "uuid",
    "workflow_step_id": "step_1",
    "content_type": "caption",
    "title": "Optional title",
    "data": {
      "tool": "caption_generator",
      "result": {
        "caption": "Generated caption text...",
        "platform": "linkedin",
        "tone": "professional",
        "length": 150
      },
      "status": "success"
    },
    "status": "draft",
    "created_at": "2025-12-26T00:00:00Z",
    "updated_at": null,
    "approved_at": null,
    "posted_at": null
  }
]
```

Get Content by ID

```
GET /api/content/{content_id}
```

Response: Single content object (same structure as list)

Update Content

```
PATCH /api/content/{content_id}
Content-Type: application/json

{
  "title": "string (optional)",
  "data": { /* updated data (optional) */ },
  "status": "draft" | "approved" | "used" | "posted" (optional)
}
```

Response: Updated content object

Approve Content (Shortcut)

```
POST /api/content/{content_id}/approve
```

Response: Content object with `status`: "approved" and `approved_at` timestamp

Delete Content

```
DELETE /api/content/{content_id}
```

Response: 200 OK

```
{
  "message": "Content deleted successfully",
  "content_id": "uuid",
  "deleted_at": "2025-12-26T00:00:00Z"
}
```

Get Pending Content

```
GET /api/content/workspace/{workspace_id}/pending?limit=20
```

Response: Array of content with `status`: "draft"

5. Generation API (Direct Tools)

Note: These are direct API endpoints for standalone generation. For automated workflows, use the Workflow API instead.

Generate Blog Post

```
POST /api/v1/generate/blog_post
Content-Type: application/json
```

```
{
  "topic": "string"
}
```

Response: 200 OK

```
{
  "id": "string",
  "topic": "AI in Marketing",
  "docx_url": "https://s3.amazonaws.com/...",
  "cover_url": "https://s3.amazonaws.com/...",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Generate Motivational Image

```
POST /api/v1/generate/motivational_post
Content-Type: application/json
```

```
{
  "topic": "string"
}
```

Response: 200 OK

```
{
  "id": "string",
  "topic": "Motivation",
  "quote_text": "Generated quote",
  "image_url": "https://s3.amazonaws.com/...",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Generate Caption

```
POST /api/v1/generate/caption
Content-Type: application/json
```

```
{
  "topic": "string",
  "platform": "linkedin",
  "tone": "professional",
  "include_emojis": true,
  "include_hashtags": true
}
```

Response: 200 OK

```
{
  "id": "string",
  "caption": "Generated caption with emojis 🚀 #hashtags",
  "platform": "linkedin",
  "tone": "professional",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Generate Hashtags

```
POST /api/v1/generate/hashtags
Content-Type: application/json
```

```
{
  "topic": "string",
  "count": 5,
  "platform": "linkedin"
}
```

Response: 200 OK

```
{
  "id": "string",
  "hashtags": ["#Marketing", "#AI", "#Automation", "#ContentCreation",
    "#Digital"],
  "count": 5,
  "platform": "linkedin",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Optimize Content

```
POST /api/v1/generate/optimize
Content-Type: application/json
```

```
{
  "content": "string",
  "goal": "engagement",
  "audience": "professionals",
  "platform": "linkedin"
}
```

Response: 200 OK

```
{
  "id": "string",
  "original": "Original content...",
  "optimized": "Improved content...",
  "goal": "engagement",
  "platform": "linkedin",
  "created_at": "2025-12-26T00:00:00Z"
}
```

Common Integration Patterns

Pattern 1: Dashboard Overview

```
async function loadDashboard(workspaceId) {
  const [workflows, recentJobs, pendingContent] = await Promise.all([
    fetch(`/api/workflows/?workspace_id=${workspaceId}`),
    fetch(`/api/jobs/workspace/${workspaceId}`),
    fetch(`/api/content/workspace/${workspaceId}/pending`)
  ]);

  return {
    workflows: await workflows.json(),
    recentJobs: await recentJobs.json(),
    pendingContent: await pendingContent.json()
  };
}
```

Pattern 2: Real-Time Job Progress

```

function JobProgressComponent({ jobId }) {
  const [job, setJob] = useState(null);

  useEffect(() => {
    const interval = setInterval(async () => {
      const response = await fetch(`/api/jobs/${jobId}`);
      const data = await response.json();
      setJob(data);

      if (data.status === 'completed' || data.status === 'failed') {
        clearInterval(interval);
      }
    }, 2000);

    return () => clearInterval(interval);
  }, [jobId]);

  return (
    <div>
      <ProgressBar value={job?.progress || 0} />
      <p>Status: {job?.status}</p>
      <p>Step: {job?.current_step}</p>
    </div>
  );
}

```

Pattern 3: Content Approval Queue

```

async function ContentApprovalQueue({ workspaceId }) {
  const pending = await fetch(
    `/api/content/workspace/${workspaceId}/pending`
  ).then(r => r.json());

  const handleApprove = async (contentId) => {
    await fetch(`/api/content/${contentId}/approve`, {
      method: 'POST'
    });
    // Refresh list
  };

  const handleReject = async (contentId) => {
    await fetch(`/api/content/${contentId}`, {
      method: 'DELETE'
    });
    // Refresh list
  };

  return (
    <div>

```

```

    {pending.map(content => (
      <ContentCard
        key={content.id}
        content={content}
        onApprove={() => handleApprove(content.id)}
        onReject={() => handleReject(content.id)}
      />
    ))}
  </div>
);
}

```

Pattern 4: Workflow Builder

```

function WorkflowBuilder({ workspaceId, onSave }) {
  const [steps, setSteps] = useState([]);

  const addStep = (toolName) => {
    setSteps([...steps, {
      order: steps.length + 1,
      name: `Step ${steps.length + 1}`,
      tool_name: toolName,
      config: getDefaultConfig(toolName)
    }]);
  };

  const saveWorkflow = async () => {
    const response = await fetch('/api/workflows/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        workspace_id: workspaceId,
        name: workflowName,
        steps: steps
      })
    });

    if (response.ok) {
      const workflow = await response.json();
      onSave(workflow);
    }
  };

  return (
    <WorkflowBuilderUI
      steps={steps}
      onAddStep={addStep}
      onSave={saveWorkflow}
    />
  );
}

```



```
    );  
}
```

Error Handling

HTTP Status Codes

- **200 OK**: Success
- **307 Temporary Redirect**: Missing trailing slash (FastAPI auto-redirects)
- **400 Bad Request**: Invalid request data
- **404 Not Found**: Resource doesn't exist
- **422 Unprocessable Entity**: Validation error
- **500 Internal Server Error**: Server error

Error Response Format

```
{  
  "detail": "Error message",  
  "errors": [  
    {  
      "loc": ["body", "field_name"],  
      "msg": "Field is required",  
      "type": "value_error.missing"  
    }  
  ]  
}
```

Error Handling Pattern

```
async function apiRequest(url, options) {  
  try {  
    const response = await fetch(url, options);  
  
    if (!response.ok) {  
      const error = await response.json();  
      throw new APIError(error.detail || 'Request failed', response.status);  
    }  
  
    return await response.json();  
  } catch (error) {  
    if (error instanceof APIError) {  
      // Handle API errors  
      console.error('API Error:', error.message);  
    } else {  
      // Handle network errors  
      console.error('Network Error:', error);  
    }  
  }  
}
```

```

    }
    throw error;
  }
}

class APIError extends Error {
  constructor(message, status) {
    super(message);
    this.status = status;
  }
}

```

TypeScript Types

```

// Enums
export enum JobStatus {
  PENDING = 'pending',
  RUNNING = 'running',
  COMPLETED = 'completed',
  FAILED = 'failed'
}

export enum ContentType {
  BLOG_POST = 'blog_post',
  IMAGE = 'image',
  CAPTION = 'caption',
  HASHTAGS = 'hashtags',
  OPTIMIZED_CONTENT = 'optimized_content'
}

export enum ContentStatus {
  DRAFT = 'draft',
  APPROVED = 'approved',
  USED = 'used',
  POSTED = 'posted'
}

export enum SocialPlatform {
  LINKEDIN = 'linkedin'
}

// Workspace
export interface Workspace {
  id: string;
  owner_user_id: string;
  name: string;
  created_at: string;
}

```

```

export interface CreateWorkspaceDto {
  owner_user_id: string;
  name: string;
}

// Workflow
export interface WorkflowStep {
  id?: string;
  order: number;
  name?: string;
  tool_name: string;
  config?: Record<string, any>;
}

export interface Workflow {
  id: string;
  workspace_id: string;
  name: string;
  description?: string;
  target_platform?: SocialPlatform;
  steps: WorkflowStep[];
  created_at: string;
  updated_at?: string;
}

export interface WorkflowListItem {
  id: string;
  name: string;
  description?: string;
  target_platform?: string;
  last_run_at?: string;
  last_run_status?: JobStatus;
  created_at: string;
}

export interface CreateWorkflowDto {
  workspace_id: string;
  name: string;
  description?: string;
  target_platform?: SocialPlatform;
  steps: Omit<WorkflowStep, 'id'>[];
}

export interface UpdateWorkflowDto {
  name?: string;
  description?: string;
  target_platform?: SocialPlatform;
  steps?: Omit<WorkflowStep, 'id'>[];
}

// Job
export interface Job {
  id: string;

```

```

    workflow_id: string;
    status: JobStatus;
    progress?: number;
    current_step?: number;
    logs?: Record<string, any>;
    started_at?: string;
    completed_at?: string;
    error?: string;
}

export interface RunWorkflowResponse {
    job_id: string;
    workflow_id: string;
    status: JobStatus;
    started_at: string;
}

// Content
export interface Content {
    id: string;
    workspace_id: string;
    job_id: string;
    workflow_step_id: string;
    content_type: ContentType;
    title?: string;
    data: Record<string, any>;
    status: ContentStatus;
    created_at: string;
    updated_at?: string;
    approved_at?: string;
    posted_at?: string;
}

export interface UpdateContentDto {
    title?: string;
    data?: Record<string, any>;
    status?: ContentStatus;
}

// Tool Configs
export interface CaptionConfig {
    topic: string;
    platform?: string;
    tone?: string;
    include_emojis?: boolean;
    include_hashtags?: boolean;
}

export interface HashtagConfig {
    topic: string;
    count?: number;
    platform?: string;
}

```

```

export interface ContentOptimizeConfig {
  content: string;
  goal?: string;
  audience?: string;
  platform?: string;
}

export interface BlogConfig {
  topic: string;
  style?: string;
}

```

Code Examples

React Hook: useWorkflow

```

import { useState, useEffect } from 'react';

export function useWorkflow(workflowId: string | null) {
  const [workflow, setWorkflow] = useState<Workflow | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    if (!workflowId) return;

    setLoading(true);
    fetch(`/api/workflows/${workflowId}`)
      .then(r => r.json())
      .then(setWorkflow)
      .catch(setError)
      .finally(() => setLoading(false));
  }, [workflowId]);

  return { workflow, loading, error };
}

```

React Hook: useJobProgress

```

import { useState, useEffect } from 'react';

export function useJobProgress(jobId: string | null) {
  const [job, setJob] = useState<Job | null>(null);

  useEffect(() => {
    if (!jobId) return;

```

```

const interval = setInterval(async () => {
  const response = await fetch(`/api/jobs/${jobId}`);
  const data = await response.json();
  setJob(data);

  if (data.status === 'completed' || data.status === 'failed') {
    clearInterval(interval);
  }
}, 2000);

return () => clearInterval(interval);
}, [jobId]);

return job;
}

```

Vue Composable: useContent

```

import { ref, computed } from 'vue';

export function useContent(workspaceId: string) {
  const content = ref<Content[]>([]);
  const loading = ref(false);

  const pending = computed(() =>
    content.value.filter(c => c.status === ContentStatus.DRAFT)
  );

  const approved = computed(() =>
    content.value.filter(c => c.status === ContentStatus.APPROVED)
  );

  async function fetchContent() {
    loading.value = true;
    try {
      const response = await fetch(
        `/api/content/?workspace_id=${workspaceId}`
      );
      content.value = await response.json();
    } finally {
      loading.value = false;
    }
  }

  async function approveContent(contentId: string) {
    await fetch(`/api/content/${contentId}/approve`, {
      method: 'POST'
    });
    await fetchContent();
  }
}

```

```

    }

    return {
        content,
        pending,
        approved,
        loading,
        fetchContent,
        approveContent
    };
}

```

Service Class Pattern

```

class MarketingHubAPI {
    constructor(private baseUrl: string) {}

    // Workspaces
    async createWorkspace(data: CreateWorkspaceDto): Promise<Workspace> {
        return this.post('/api/workspaces/', data);
    }

    async getWorkspace(id: string): Promise<Workspace> {
        return this.get(`/api/workspaces/${id}`);
    }

    // Workflows
    async createWorkflow(data: CreateWorkflowDto): Promise<Workflow> {
        return this.post('/api/workflows/', data);
    }

    async listWorkflows(workspaceId: string): Promise<WorkflowListItem[]> {
        return this.get(`/api/workflows/?workspace_id=${workspaceId}`);
    }

    async getWorkflow(id: string): Promise<Workflow> {
        return this.get(`/api/workflows/${id}`);
    }

    async updateWorkflow(id: string, data: UpdateWorkflowDto): Promise<Workflow>
    {
        return this.put(`/api/workflows/${id}`, data);
    }

    async deleteWorkflow(id: string): Promise<void> {
        return this.delete(`/api/workflows/${id}`);
    }

    async runWorkflow(id: string): Promise<RunWorkflowResponse> {
        return this.post(`/api/workflows/${id}/run`);
    }
}

```

```

}

// Jobs
async getJob(id: string): Promise<Job> {
  return this.get(`/api/jobs/${id}`);
}

async getWorkflowJobs(workflowId: string): Promise<Job[]> {
  return this.get(`/api/jobs/workflow/${workflowId}`);
}

async waitForJob(
  jobId: string,
  onProgress?: (job: Job) => void
): Promise<Job> {
  return new Promise((resolve, reject) => {
    const interval = setInterval(async () => {
      try {
        const job = await this.getJob(jobId);
        onProgress?.(job);

        if (job.status === 'completed') {
          clearInterval(interval);
          resolve(job);
        } else if (job.status === 'failed') {
          clearInterval(interval);
          reject(new Error(job.logs?.error || 'Job failed'));
        }
      } catch (error) {
        clearInterval(interval);
        reject(error);
      }
    }, 2000);
  });
}

// Content
async listContent(params: {
  workspace_id?: string;
  job_id?: string;
  content_type?: ContentType;
  status?: ContentStatus;
  limit?: number;
}): Promise<Content[]> {
  const query = new URLSearchParams(
    Object.entries(params)
      .filter(([, v]) => v !== undefined)
      .map(([k, v]) => [k, String(v)])
  );
  return this.get(`/api/content/?${query}`);
}

async getContent(id: string): Promise<Content> {

```



```

    return this.get(`/api/content/${id}`);
}

async updateContent(id: string, data: UpdateContentDto): Promise<Content> {
    return this.patch(`/api/content/${id}`, data);
}

async approveContent(id: string): Promise<Content> {
    return this.post(`/api/content/${id}/approve`);
}

async deleteContent(id: string): Promise<void> {
    return this.delete(`/api/content/${id}`);
}

async getPendingContent(workspaceId: string): Promise<Content[]> {
    return this.get(`/api/content/workspace/${workspaceId}/pending`);
}

// HTTP helpers
private async request<T>(
    url: string,
    options: RequestInit
): Promise<T> {
    const response = await fetch(`${this.baseURL}${url}`, {
        ...options,
        headers: {
            'Content-Type': 'application/json',
            ...options.headers
        }
    });

    if (!response.ok) {
        const error = await response.json();
        throw new APIError(error.detail || 'Request failed', response.status);
    }

    return response.json();
}

private get<T>(url: string): Promise<T> {
    return this.request(url, { method: 'GET' });
}

private post<T>(url: string, data?: any): Promise<T> {
    return this.request(url, {
        method: 'POST',
        body: data ? JSON.stringify(data) : undefined
    });
}

private put<T>(url: string, data: any): Promise<T> {
    return this.request(url, {

```

```

        method: 'PUT',
        body: JSON.stringify(data)
    });
}

private patch<T>(url: string, data: any): Promise<T> {
    return this.request(url, {
        method: 'PATCH',
        body: JSON.stringify(data)
    });
}

private delete<T>(url: string): Promise<T> {
    return this.request(url, { method: 'DELETE' });
}
}

// Usage
const api = new MarketingHubAPI('http://localhost:8000');

```

Testing Tips

1. Test Workflow Execution End-to-End

```

describe('Workflow Execution', () => {
    it('should create and run workflow successfully', async () => {
        // Create workflow
        const workflow = await api.createWorkflow({
            workspace_id: testWorkspaceId,
            name: 'Test Workflow',
            steps: [{
                order: 1,
                tool_name: 'caption_generator',
                config: { topic: 'test' }
            }]
        });

        // Run workflow
        const job = await api.runWorkflow(workflow.id);
        expect(job.status).toBe('pending');

        // Wait for completion
        const completedJob = await api.waitForJob(job.job_id);
        expect(completedJob.status).toBe('completed');

        // Verify content created
        const content = await api.listContent({ job_id: job.job_id });
        expect(content.length).toBeGreaterThan(0);
        expect(content[0].status).toBe('draft');
    });
});

```

```
});  
});
```

2. Mock API Responses

```
// Mock data for development  
export const mockWorkflow: Workflow = {  
  id: 'mock-uuid',  
  workspace_id: 'workspace-uuid',  
  name: 'Test Workflow',  
  steps: [  
    {  
      id: 'step_1',  
      order: 1,  
      name: 'Generate Caption',  
      tool_name: 'caption_generator',  
      config: { topic: 'test' }  
    }  
  ],  
  created_at: new Date().toISOString()  
};  
  
export const mockJob: Job = {  
  id: 'job-uuid',  
  workflow_id: 'workflow-uuid',  
  status: JobStatus.COMPLETED,  
  progress: 100,  
  started_at: new Date().toISOString(),  
  completed_at: new Date().toISOString()  
};
```

Performance Considerations

1. Polling Optimization

- Use exponential backoff for failed requests
- Stop polling after reasonable timeout (5-10 minutes)
- Use WebSockets for real-time updates (future enhancement)

2. Caching Strategy

```
// Cache workflow list  
const workflowCache = new Map();  
  
async function getWorkflowsCached(workspaceId, maxAge = 60000) {  
  const cached = workflowCache.get(workspaceId);
```

```

    if (cached && Date.now() - cached.timestamp < maxAge) {
      return cached.data;
    }

    const data = await api.listWorkflows(workspaceId);
    workflowCache.set(workspaceId, {
      data,
      timestamp: Date.now()
    });

    return data;
  }

```

3. Pagination

```

// Implement infinite scroll for content
async function loadMoreContent(workspaceId, offset = 0) {
  const limit = 20;
  const content = await api.listContent({
    workspace_id: workspaceId,
    limit
  });

  return {
    items: content,
    hasMore: content.length === limit
  };
}

```

Migration Guide (Future Updates)

Adding Authentication Headers

When authentication is fully implemented:

```

// Add to all requests
const headers = {
  'Content-Type': 'application/json',
  'Authorization': `Bearer ${accessToken}`
};

```

WebSocket Support (Planned)

```
// Future: Real-time job updates
const ws = new WebSocket('ws://localhost:8000/ws/jobs');

ws.onmessage = (event) => {
  const job = JSON.parse(event.data);
  updateJobUI(job);
};
```

Support & Resources

- **API Documentation:** <http://localhost:8000/docs>
- **Backend README:** See [README.md](#) in backend-python folder
- **Workflow Guide:** See [README_WORKFLOW_APIS.md](#)
- **Content Layer Guide:** See [README_CONTENT_LAYER.md](#)

Changelog

Version 1.0.0 (December 26, 2025)

- ☒ Complete workflow orchestration system
- ☒ Job execution with progress tracking
- ☒ Content management with approval workflows
- ☒ AI-powered content generation tools
- ☒ Real-time job status polling
- ☒ Soft delete support
- ☒ JSONB flexible content storage

Questions? Check the interactive API docs at </docs> or contact the backend team.